

Technical Specification — Chat & Social Web Application

Implementation blueprint · Version 0.1 · Last updated July 11, 2026 · Derived from Requirement Scope v0.3

Cost constraint carried forward: every component below runs on a permanently-free tier or self-hosted-free option. Where a provider is named, it is a recommendation, not a lock-in — any equivalent free-tier service can be substituted without changing the architecture.

PART I — STACK & ARCHITECTURE

1. Technology Stack

Layer	Choice	Free-tier basis
Frontend	React 18 + Vite + TypeScript, Tailwind CSS	Open-source, no runtime license cost
Offline/PWA	Service worker via Workbox	Browser-native API, no service dependency
Backend API + sockets	Node.js + Express + TypeScript, Socket.IO	Self-hosted, no per-connection billing
Database	PostgreSQL (Supabase free project)	500 MB DB + 1 GB file storage, free forever
ORM	Prisma	Open-source
Auth	Custom JWT (access + refresh) + Argon2id hashing	No paid identity vendor (Auth0, etc.)
In-app cache	In-process LRU (node-cache)	Avoids paid Redis tier (Section 20 requirement)
Media storage/CDN	Cloudinary free tier	25 monthly credits, built-in transform/compression
Transactional email	Brevo free plan	300 emails/day — verification, magic link, OTP, new-device confirm
CAPTCHA	Cloudflare Turnstile	Unconditionally free, unlimited
Push notifications	Web Push API (VAPID keys)	Browser-native, no FCM/APNs cost
Calls / live (signaling)	Socket.IO channel (existing realtime layer)	No separate signaling service
Calls / live (STUN)	Google public STUN (stun.l.google.com)	Free, no account needed
Calls / live (TURN)	Self-hosted coturn on Oracle Cloud Always-Free VM	Genuinely free forever, not a trial
Frontend hosting	Vercel free tier	Static + edge, generous free bandwidth
API/socket hosting	Render free web service	Free persistent Node process (sleeps when idle)
CI/CD	GitHub Actions	Free minutes on free/public repos
Uptime monitoring	UptimeRobot free plan	50 monitors free

Analytics	Self-hosted event table + Recharts dashboard	No paid BI/analytics tool
DB backups	GitHub Actions cron → pg_dump → Backblaze B2	2,000 free CI minutes/mo + 10 GB free storage
Status background music	CC0 tracks from Free Music Archive, bundled as static assets	No licensing fee
Profanity/spam filter	Open-source wordlist (e.g. "bad-words" npm package)	No paid moderation API

2. System Architecture

Four independently-deployed, all-free components:

React SPA (Vercel) → HTTPS/WSS → Node API + Socket.IO server (Render) → Postgres + file storage (Supabase) | peer WebRTC media negotiated via Socket.IO signaling, relayed through self-hosted coturn (Oracle VM) when a direct peer connection isn't possible

- The SPA is a single build; there is no separate admin frontend — the admin dashboard (Section 13) is a route gated by an admin-only JWT claim.
- Socket.IO runs in the same Node process as the REST API so both share the request-scoped user/session lookup and the in-process cache.
- Media files (attachments, status images, profile photos) never pass through the API server — the client uploads directly to Cloudinary using a short-lived signed upload token issued by the API.
- The database is the single source of truth for message state (Section 21.2); Socket.IO only carries live notifications of state that is already, or about to be, persisted.

3. Repository Structure

```
apps/
  web/      React SPA (Vite, TS, Tailwind)
  api/      Express + Socket.IO server (TS)
packages/
  shared/   Shared TS types, validation schemas (zod), crypto helpers
infra/
  coturn/   TURN server config for the Oracle VM
  github-actions/ CI + scheduled backup workflows
prisma/
  schema.prisma
```

```
migrations/  
seed.ts
```

PART II — DATA & DOMAIN MODEL

4. Core Entities

Entity	Key fields	Notes
User	id, username, display_name, password_hash, email, email_verified, birth_date, country, state, bio, avatar_url, visibility (public/friends/private), role (user/admin), status (active/deactivated/deleted), created_at	Soft-delete via status, never hard-erased (Section 16)
PrivacySetting	user_id, who_can_send_requests, email_visible, birthdate_visible, last_seen_visibility, status_visibility	1:1 with User
Device / Session	id, user_id, refresh_token_hash, device_fingerprint, user_agent, recognized (bool), last_seen_at, revoked_at	Backs Sections 6.4–6.6 and 16.1
FriendRequest	id, from_user_id, to_user_id, status (pending/accepted/rejected/cancelled), created_at	Capped at 20 pending outgoing per user (Section 20)
Friendship	user_a_id, user_b_id, created_at	Symmetric row created on accept
Block	blocker_id, blocked_id, created_at	One-directional (Section 10.2)
Conversation	id, type (direct/group), name (group only), created_at	Direct conversations require an underlying Friendship
ConversationMember	conversation_id, user_id, role (member/admin), joined_at, wrapped_key (Section 6)	wrapped_key implements per-member envelope encryption
Message	id, conversation_id, sender_id, ciphertext, nonce, reply_to_id, forwarded_from_id, edited_at, deleted_for_everyone_at, sequence, client_uuid, created_at	sequence + client_uuid give ordering & idempotency (Section 21.2)

MessageStatus	message_id, user_id, state (notified/delivered/read), updated_at	Per-recipient row; powers 14.1/14.2 breakdowns
MessageReaction / Star	message_id, user_id, emoji (reaction only)	Star has no emoji column, just presence
Status	id, user_id, media_url, caption, music_track_id, visibility, expires_at	expires_at = created_at + 24h; swept by a cron job
LiveSession	id, user_id, visibility, started_at, ended_at	Row only exists while broadcast is active/just ended
Post	id, author_id, media_url, caption, like_count, comment_count, view_count, created_at	Denormalized counters, kept in sync via transaction
Hashtag / PostHashtag	tag (unique), post_id	Ranking score = f(likes, comments, avg views) computed at read time
Comment / Like / Save	post_id, user_id, body (comment only), created_at	Save is private, never notifies author (13.5)
Report	id, reporter_id, target_type (post/message/profile), target_id, reason, status (open/reviewed/actioned), created_at	Feeds admin moderation queue (Section 18)
Notification	id, user_id, type, payload_json, read_at, created_at	Drives in-app + push (Section 17)
AuditLogEntry	id, user_id, event_type, ip, device, created_at	Owner-visible security log (Section 20)
AnalyticsEvent	id, event_type, user_id (nullable), created_at	Raw event stream aggregated for the admin dashboard (Section 18.1)

Indexes: users(username), messages(conversation_id, sequence), posthashtag(tag), friendrequest(to_user_id, status), analyticsevent(event_type, created_at) — ties to Section 20 performance requirement.

PART III — SECURITY-CRITICAL DESIGN

5. Authentication & Session Design

- Passwords hashed with Argon2id (memory-hard, free, no external service).
- Login issues a short-lived JWT access token (~15 min) plus a rotating refresh token stored hashed against the Device/Session row — enables per-device revocation (Section 6.5).
- Magic link: a single-use, time-boxed token emailed via Brevo; opening it exchanges the token for a session, same code path as password login post-verification.
- Email OTP 2FA: 6-digit code emailed via Brevo, checked server-side with attempt limiting.
- New-device flow: device fingerprint (hashed UA + coarse network signal) checked against known Device rows for the account; unrecognized fingerprint creates a "pending" session, emails a confirm link, and only marks the device recognized once clicked (Section 6.6).
- Brute-force protection: exponential backoff + temporary lockout keyed by username+IP, tracked in the same in-process cache used for rate limiting.

6. Encryption-at-Rest Design

Implements Section 20's requirement that message content never be readable server-side, including via direct database access.

- At signup, the client generates an X25519 keypair in-browser (libsodium). The private key is encrypted with a key derived from the user's password (Argon2id, client-side) and stored in IndexedDB; only the public key is sent to the server.
- Each conversation has one AES-256-GCM content key, generated client-side when the conversation is created.
- That content key is wrapped (encrypted) separately for every member using their X25519 public key, and the wrapped copies are stored in ConversationMember.wrapped_key — this is why the server can distribute a conversation to N members without ever holding the plaintext key.
- Every message body is encrypted client-side before send; the server stores and relays ciphertext + nonce only.
- In-chat search (Section 14.12) runs entirely client-side over the conversation's already-decrypted, locally-cached messages — the server is never asked to search plaintext.
- Trade-off, stated plainly: losing the password with no recovery email means the private key (and history) cannot be recovered — acceptable for a portfolio-scale project and disclosed in the Privacy Policy.

7. Security Controls Checklist

- Input validation on every endpoint via shared zod schemas (packages/shared)
- Parameterized queries only (Prisma) — no raw SQL string concatenation, closes SQL injection

- Output encoding + React's default escaping closes reflected/stored XSS; rich text fields are sanitized server-side (e.g. `sanitize-html`) before storage
- CSRF: `SameSite=Strict` cookies for refresh token; access token sent as a Bearer header, not a cookie, so classic CSRF doesn't apply to it
- Rate limiting via `express-rate-limit` backed by the in-process cache, scoped per user/IP per endpoint class (login, message send, friend request)
- Cloudflare Turnstile on signup and login forms
- HTTPS/WSS everywhere — free TLS from Vercel/Render/Let's Encrypt on the TURN VM
- Profanity/spam wordlist filter applied to messages and post captions before storage
- Security audit log (`AuditLogEntry`) surfaced read-only in Settings → Security

PART IV — INTERFACES

8. REST API Surface

Group	Representative endpoints
Auth	POST /auth/register · POST /auth/login · POST /auth/magic-link · POST /auth/magic-link/verify · POST /auth/otp/verify · POST /auth/refresh · POST /auth/logout · GET /auth/devices · DELETE /auth/devices/:id
Profile	GET /users/:username · PATCH /users/me · PATCH /users/me/privacy · POST /users/me/avatar-upload-token
Search & Friends	GET /search/users?q= · POST /friend-requests · PATCH /friend-requests/:id · GET /friends · GET /friends/suggestions · POST /blocks · DELETE /blocks/:id
Invites	POST /invites · GET /invites/:code
Conversations	GET /conversations · POST /conversations · GET /conversations/:id/messages?cursor= · PATCH /conversations/:id (pin/mute/archive)
Messages	POST /messages · PATCH /messages/:id · DELETE /messages/:id?scope=me\ everyone · POST /messages/:id/reactions · POST /messages/:id/star
Status & Live	POST /statuses · GET /statuses/feed · POST /live/start · POST /live/end · GET /live/active
Posts & Feed	POST /posts · GET /posts/:id · GET /hashtags/:tag?cursor= · GET /feed/explore?cursor= · POST /posts/:id/like · POST /posts/:id/comments · POST /posts/:id/save
Notifications	GET /notifications · PATCH /notifications/:id/read · POST /push/subscribe
Moderation	POST /reports · GET /admin/reports · PATCH /admin/reports/:id
Admin analytics	GET /admin/analytics/summary · GET /admin/analytics/timeseries?metric=&range=
Account	PATCH /account/password · POST /account/deactivate · POST /account/delete · POST /account/restore · GET /account/export
System	GET /healthz

All list endpoints accept a cursor + limit and return {items, nextCursor}. All responses expose X-RateLimit-* headers.

9. Real-Time Event Catalog (Socket.IO)

Direction	Event	Purpose
Client → Server	message:send	Carries client_uuid + ciphertext; server persists, assigns sequence, then fans out
Client → Server	message:ack	Recipient acknowledges delivered/read → written to MessageStatus
Client → Server	typing:start / typing:stop	Section 14.10 typing indicator
Client → Server	presence:heartbeat	Keeps last-seen fresh
Client → Server	call:offer / call:answer / call:ice-candidate	WebRTC signaling relay for calls and live
Server → Client	message:new / message:edited / message:deleted	Live fan-out to conversation members' sockets only (Sections 10, 15)
Server → Client	message:status	Per-recipient delivered/read update, matches DB (Section 21.1)
Server → Client	presence:update	Online/offline + last-seen, filtered by each viewer's visibility rights
Server → Client	notification:new	Friend request, acceptance, activity alerts (Section 17)
Server → Client	active-count:update	Live active-user counter (Section 12.2)

On connect, the client sends its last known sequence per open conversation; the server replays any gap from Postgres before resuming live events — the reconciliation step required by Section 21.2.

10. Media & File Handling

- Client requests a signed Cloudinary upload token from the API (scoped, short-lived), then uploads directly — the API server never proxies file bytes.
- Images are downscaled/compressed client-side (browser-image-compression) before upload; all file types are hard-capped at 10 MB client-side and re-checked via Cloudinary's upload preset limit.
- Attachment picker (Document/Image/Video/Audio/Camera) all resolve to the same signed-upload path with a type-specific Cloudinary resource_type.

- Image annotation (drawing/stickers) happens client-side on a canvas before the compressed result is uploaded.

11. Live Streaming & Calling

- 1:1 voice/video calls: direct peer-to-peer WebRTC, STUN first, coturn TURN relay as fallback when direct connection fails (symmetric NAT, etc).
- Live broadcast: mesh WebRTC (broadcaster connects directly to each viewer) — appropriate at small/demo audience sizes; documented as the scaling ceiling of a free-tier, single-TURN-VM setup rather than a hidden limitation.
- coturn runs as a systemd service on the Oracle Always-Free VM, config in `infra/coturn/`, credentials issued short-lived via the API using the turnserver shared-secret scheme (no static creds baked into the client).

12. Notifications

- In-app: Notification rows rendered from a bell menu, marked read on view.
- Push: client subscribes via the browser Push API with a VAPID public key; API stores the subscription and pushes via web-push (npm) directly — no FCM/APNs account needed.
- Covers: friend request received/accepted, new message, account activity alerts (Section 17).

13. Content Moderation & Admin Analytics

- Reports (post/message/profile) land in Report with status open; admin console lists open reports, actions = warn/remove-content/suspend/dismiss.
- Admin dashboard reads only AnalyticsEvent aggregates and Report rows — schema-level, the admin role has no query path to Message.ciphertext plaintext, matching Section 18.1's isolation guarantee.
- Metrics: active-now (from presence), DAU/WAU (distinct AnalyticsEvent user_ids per window), total users + growth, traffic over time — charted with Recharts, selectable date range.

PART V — DELIVERY

14. Performance & Caching Strategy

- Every list endpoint is cursor-paginated (id/sequence-based, not offset) — feed, hashtag pages, search results, chat history.
- Responses are field-scoped per view (no over-fetching) via Prisma select.
- In-process LRU caches hot, frequently-read data (profile summaries, hashtag pages) with short TTLs; invalidated on write.

- Search-as-you-type and typing indicators are debounced/throttled client-side (~250–300ms).
- Postgres indexes as listed in Section 4's footnote.

15. Free-Tier Service Directory

Purpose	Provider	Free limit	If exceeded
Database + storage	Supabase	500 MB DB / 1 GB storage	Prune old media, archive AnalyticsEvent
Media CDN	Cloudinary	25 credits/mo	Lower default image quality/size
Email	Brevo	300 emails/day	Queue and drip-send, or add a 2nd free provider
Frontend hosting	Vercel	100 GB bandwidth/mo	Switch to Netlify/Cloudflare Pages free tier
API hosting	Render	750 hrs/mo, sleeps when idle	Fly.io free allowance as alternate host
TURN relay	Self-hosted coturn (Oracle Always-Free VM)	Bandwidth of the free VM tier	Fall back to STUN-only (fails behind strict NAT)
CI/CD	GitHub Actions	2,000 min/mo (private repos)	Trim workflow frequency
Backups	Backblaze B2	10 GB storage	Reduce retention window
Uptime checks	UptimeRobot	50 monitors	N/A — well within limit

16. Deployment & CI/CD

- GitHub Actions on push to main: lint → typecheck → unit tests → build → deploy web (Vercel) and api (Render) via their free Git-integration deploy hooks.
- Separate scheduled workflow (nightly cron): pg_dump the Supabase database, upload to Backblaze B2 (Section 21 backup requirement).
- Environment-based config: .env.development / .env.production, no secret committed to source; secrets live in each host's free environment-variable store.
- UptimeRobot polls GET /healthz every 5 minutes.

17. Environment Variables

Variable	Purpose
DATABASE_URL	Supabase Postgres connection string
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	Token signing
BREVO_API_KEY	Transactional email
CLOUDINARY_URL	Signed upload tokens
TURNSTILE_SECRET	CAPTCHA verification
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY	Web Push
TURN_SHARED_SECRET	Short-lived TURN credential issuance
APP_ORIGIN	CORS + link generation (magic links, invites)

18. Build Sequence

Internal implementation order — the release itself ships as one unit (Requirement Scope Section 22); this is only the order code gets written in.

- **M0** — repo scaffold, Prisma schema, CI pipeline, health check
- **M1** — auth (register/login/JWT/sessions), profile, privacy settings
- **M2** — search, friend requests, blocking, invites
- **M3** — messaging core: Socket.IO wiring, encryption, send/receive, ordering/reconciliation, read receipts
- **M4** — messaging polish: edit/delete, reactions, reply/forward, starred, attachments, in-chat search
- **M5** — status updates, live streaming, presence/active-count
- **M6** — posts, hashtags, feed ranking, explore, mentions
- **M7** — notifications (push + in-app), account management, data export, moderation queue, admin analytics
- **M8** — non-functional pass: rate limiting, caching, offline service worker, empty states/skeletons, accessibility, error pages, seed/demo data

19. Testing Strategy

- Unit tests: Vitest, covering validation, encryption helpers, ranking algorithm
- API integration tests: Supertest against a test Postgres instance (Docker locally, ephemeral in CI)
- End-to-end: Playwright (free), covering signup → friend request → chat → post happy paths

- Load sanity check on Socket.IO fan-out before demo, using an open-source tool (artillery) — not a paid load-testing SaaS

20. Appendix: Seed & Demo Data

prisma/seed.ts creates a small connected graph for local dev and demoing: ~15 users across all three privacy levels, friendships, a handful of conversations with sample encrypted messages, a couple of posts per hashtag bucket, and one open report — enough to exercise every screen without hitting free-tier storage limits.